

Real-Time Systems - Laboratory Exercise 5 (week 6)

Lab Note: These tasks are to be conducted individually (but you should discuss with your friend, don't get lost alone). Although you don't need to submit a report for assessment as in Melbourne, you should make a **note document** and keep your solution/s **code (*.c file/s)**, so that you can review later when doing the final project, as well as prepare for the In-class Tests.

In the Note document, you should have the required diagrams, screen captures and text to answer the questions. Your note should also contain examples for different input and expected output to demonstrate the behaviour of your program meets the outlined objective. Finally, keep the source files (*.c files) together with the note.

1. State Machine Example in C

Consider the following state machine:

In C this can be coded as:

```

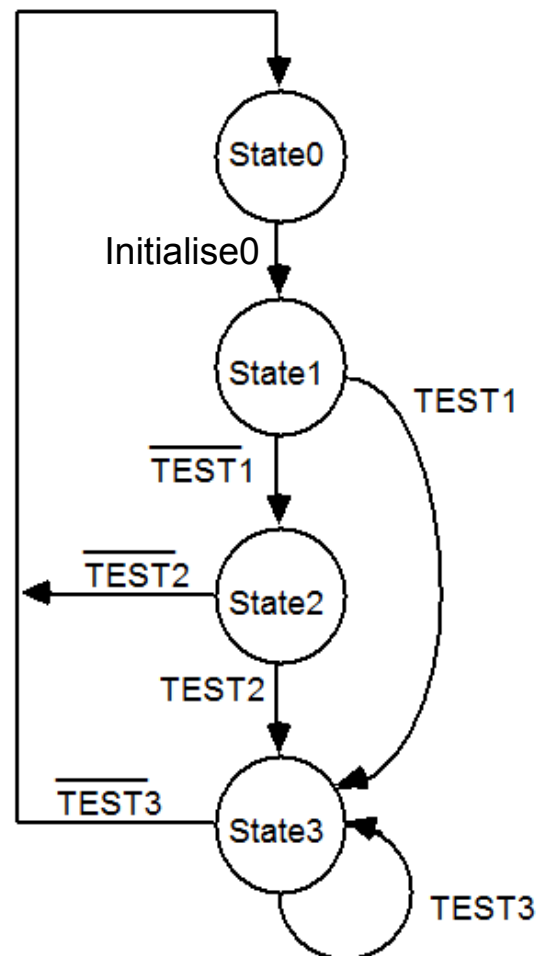
-----
enum states { State0,
              State1,
              State2,
              State3 };
enum states CurState;
-----

int Test1();
int Test2();
int Test3();
-----

void Initialise0();
void DoSomething1();
void DoSomething2();
void DoSomething3();
-----

// following in main
while (!Done)
{
    switch (CurState)
    {
        case State0:
            Initialise0(); // code or function call
            CurState = State1;
            break;
        case State1:
            DoSomething1(); // code or function call
            if (Test1())
                CurState = State3;
            else
                CurState = State2;
            break;
        case State2:
            DoSomething2(); // code or function call

```



```

        if (Test2())
            CurState = State3;
        else
            CurState = State0;
        break;

    case State3:
        DoSomething3(); // code or function call
        if (!Test3())
            CurState = State0;
        break;
    }
}

```

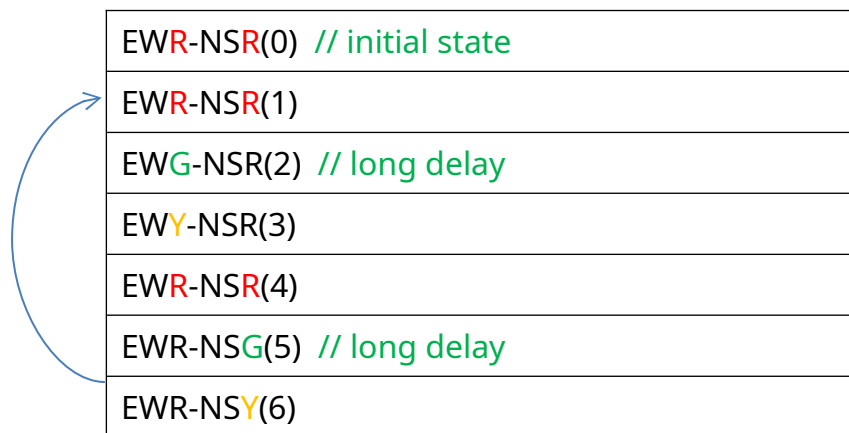
Note that looping is handled by the outer while loop. The individual cases of the state machine should not have any loops that wait for a condition to change. Similarly, any functions called by a case statement must not have any loops. By following these rules, it is possible to embed more than one state machine within another state machine.

TASK 1A: Start a new QNX Momentics project and use code outlined on the previous page to create a simple state machine that prints to the console terminal the current state and prompts the user if the test should be true or false.

2. Implementing Traffic Lights Using a State Machine

TASK 2A: Based on the code you developed in Task 1A, write a state machine that could be used to control traffic lights at a crossroad intersection with fixed timing.

The suggested traffic light sequence is:



where EWG means East and West traffic lights are green and NSR means the North and South traffic lights are Red (Y means yellow, aka amber). Use printf() statements to show the current state of the lights each time they change.

Typically the lights for the green state will be longer than the delay time used for the Yellow and Red-Red times, therefore instead of having user input to trigger the change between each state, just call the function sleep() to cause a 1 second delay for yellow and red states and 2 second for the green state. When implementing the code, it is suggested that the state machine case statement is placed in a function which can be called from the main, hence the code in the main could look like:

```
int main(int argc, char *argv[])
```

```

{
    printf("Fixed Sequence Traffic Lights State Machine Example\n");

    int Runtimes=30, counter = 0;
    enum states CurrentState; // Declaring the enum within the main
                             // means we will need to pass it by address
    while (counter < Runtimes)
    {
        SingleStep_TrafficLight_SM( &CurrentState ); // pass address
        counter++;
    }
}

```

TASK 2B: To make the traffic lights more realistic, take the code from Task 2A, and implement a sensor driven traffic lights state machine. A sensor driven traffic lights means that if the lights are currently in the EWG-NSR state, they stay in that state until a car stops at the North or South red lights and triggers the North or South car sensor, in which case the lights will cycle from EWG-NSR through EWY-NSR to EWR-NSR, and then to EWR-NSG. Then the lights will stay in the EWR-NSG state until a car stops at the East or West red lights and triggers the East or West car sensor, in which case the lights will cycle from EWR-NSG through EWR-NSY to EWR-NSR and then to EWG-NSR.

Simulate the car sensors by displaying a prompt the user to type a character on the keyboard (similar to Task 1A). In this case you can use 'e' to represent a car waiting at the East-West road and 'n' for the North-South road.

TASK 2C: As you would have noticed, reading from the keyboard blocks the thread or process until a key is typed. The safe way to read a character from a keyboard (in a real-time system) is to use a separate thread or process to read the keyboard and communicate with the state machine via a shared variable or a message respectively. Modify the code from Task 2B to implement a separate thread to read the keyboard and store the sensor data in a shared variable that can be used by the state machine loop. This enables the state machine to act on the sensor data independently from the thread that is in charge of getting (or generating) the sensor data.

In your report please provide the code snippets showing how you have implemented the key reader thread.

TASK 2D: Change the program from Task 2C above to read the keyboard in a separate process and send a message to the state machine process through a message queue. The state machine process should also use a thread to wait for the message (containing the sensors data) and provide the sensor data to the state machine via a shared variable (which will most likely be a struct of some sort). You may want to use a MESSAGE_SIZE of size 2 (one for the character e or n and the second character for the \0 delimiter. Also Remember to include the library: `#include <queue.h>`

In your report please provide the code snippets showing how you have modified the key reader thread to send the data via mqueue to the state machine thread. You are also required to submit your functional solution as a C file. To gain full marks for this task, ensure that the state machine single step drive function is called from main (i.e. the switch/case statement is in its own function) and you are using pointers to pass the current state between functions. Therefore, the following line should appear in the main:

```
enum states CurrentState = State0;
```

IMPORTANT: The code you generate for Lab 4 TASK 2D will be used for a submission task in Lab 5 as well. Make sure you keep the code safe and that it is commented well.

3. QNX Periodic Timer

QNX implements POSIX timers. Timers can be periodic (as in the supplied [timer_per1.c](#) example) or one-shot. POSIX timers can send a standard UNIX style signal when they expire, or they can be used create and launch a thread.

QNX also supports a non-POSIX extension that allows the timer to send a pulse when the timer expires (as in the example). The kind of timer created is a combination of arguments used when invoking the `timer_settime()` function. More information can be found in the QNX help files: http://www.qnx.com/developers/docs/7.0.0/index.html#com.qnx.doc.neutrino.lib_ref/topic/t/timercreate.html

The process of setting up a timer involves using communications channels to receive messages/pulses from a per-process timer that runs from a specified clock source. This removes the need for using `sleep()` type functions within a process, rather your process will block until it receives a message from the timer.

TASK 3A: Download the [timer_per1.c](#) file from Canvas and make a new QNX Momentics project. This is an example of how to implement a simple state machine with periodic timing. Compile and run the program and make sure that you understand the program, especially the library functions used, by looking up the functions in the QNX website or in the provided PDF reference files .

Once you are happy that you understand how the QNX implements POSIX timers, try adding a second `itimerspec` struct and set the `it_value` and `it_interval` times to 5 seconds. By creating a second `itimerspec` struct you should be able to set the timer with either 1.5 second delay or a 5 second delay. State 1 should be set to have a 5 second delay and states 0 and 2 should have a 1.5 second delay. You can do this by re-initialising the `itimerspec` struct and calling `timer_settime()` within the case statements in the state machine loop:

```
itime_B.it_value.tv_sec = 5;    // 5 second
itime_B.it_value.tv_nsec = 0;  //
itime_B.it_interval.tv_sec = 5; // 5 second (0 will make it single shot)
itime_B.it_interval.tv_nsec = 0; //
timer_settime(timer_id, 0, &itime_B, NULL); // Start the timer!
```

The `it_value` specifies how long from now the timer should go off and the `it_interval` value specifies a the reload time so that the timer can periodically trigger. Note that specifying a value of zero for the `it_interval` makes it into a one-shot timer (i.e. It will not trigger periodically).

TASK 3B: Change the fixed sequence traffic lights program of Task 2A to use a periodic timer (e.g. of 1 second) to control the delay for yellow and red states and a 2 second delay or the green state. Your code should not have any `usleep()` or `sleep()` type functions in your program as the delays should be provided by the periodic timers.

Your coded solution for task 3B needs to be uploaded as a functional *.c file.

What kind of timer are available? If you want to know more about QNX timers, why not have a read of this: http://www.qnx.com/developers/docs/7.0.0/index.html#com.qnx.doc.neutrino.getting_started/topic/s1_timer.html